



البرمجة بلغة ++C

جنى الخطاب

User-Defined Functions II

Void Functions

- Void functions and value-returning functions have similar structures.
- Both have a heading part and a statement part.

هذا النوع void functions يشبه value-returning function في الشكل، الاثنان الهم heading و statement، لكن هذا النوع ينفذ جمل عند استدعائه بدون أن يرجع بقيمة.

- User-defined void functions can be placed either before or after the function main.
- If user-defined void functions are placed after the function main, the function prototype must be placed before the function main.

كما هو الحال مع value-returning function ممكن أن يكون الfunction معرف قبل أو بعد الmain function لكن في حال كان معرف بعده يجب أن يوضع الprototype الخاص به قبل الmain.

- A void function does not have a return type, **return** statement without any value is typically used to exit the function early.

void function لا يرجع أي قيمة لكن ممكن أن تستخدم **return** داخل الfunction فقط للخروج من الfunction عند شرط معين وعدم تنفيذ الباقي من الجمل.

- Formal parameters are optional.
- A call to a void function is a stand-alone statement.

الparameters اختيارية مش شرط تكون موجودة، واستدعاء الfunction يكون في جملة وحدها عكس الvalue-returning function لأنه لا يرجع بقيمة فلا أستطيع أن استخدم الfunction call في assignment statement مثلاً.

Void Functions without Parameters

- Function definition syntax:

```
void functionName()  
{  
    statements  
}
```

- **void** is a reserved word.
- Function call syntax:

```
functionName ();
```

يشبه ال value-returning function الا ان نوعه void.

Void Functions with Parameters

- Function definition syntax:

```
void functionName(formal parameter list)
{
    statements
}
```

- Formal parameter list syntax:

```
dataType& variable, dataType& variable, ...
```

- Function call syntax:

```
functionName(actual parameter list);
```

- Actual parameter list syntax:

```
expression or variable, expression or variable, ...
```

الفرق الوحيد هو نوعه وانه لا يرجع قيمة.

- Value parameter: a formal parameter that receives a copy of the content of corresponding actual parameter.

الfunction يستقبل ال parameter الذي نرسله اليه ويأخذ نسخة منه.

- Reference parameter: a formal parameter that receives the location (memory address) of the corresponding actual parameter.

الfunction يستقبل ال memory location الخاص ب الparameter الذي أرسلناه يعني يأخذ ال variable بنفسه ليس نسخة منه (يصبح الreference parameter هو نفسه الactual parameter)، وإذا أجرى عليه تغييرات يظهر أثرها في الmain function.

بحب أشبههم بال current location و ال live location، اذا كان value parameter مثل ان تشارك موقعك الحالي مع شخص على google maps شو ما تحركت انت بعدها بضل ال location عنده نفس الاشياء، اما اذا كان reference parameter مثل ان تشارك live location مع شخص، شو ما تتنقل يظهر عند الشخص التغييرات.

- Reference parameters can:
 - Pass one or more values from a function.
 - Change the value of the actual parameter.

ال reference parameters يمكنك من ارسال قيمة أو أكثر وتغيير قيمة ال actual parameter.

- Reference parameters are useful in three situations:
 - Returning more than one value.

مثال:

```
#include <iostream>
using namespace std;

void calculate(int a, int b, int &sum, int &product) {
    sum = a + b;
    product = a * b;
}

int main() {
    int num1 = 3, num2 = 4;
    int sum, product;
    calculate(num1, num2, sum, product);
    cout << "Sum: " << sum << ", Product: " << product << endl;
    return 0;
}
```

في هذا المثال جعلت قيمة sum و product يتم حسابها في ال calculate function ثم رجعتها الى ال main function وطبعتها. (هذا لا يمكن تطبيقه باستخدام value-returning function لانهم قيمتين).

ملاحظة: ممكن أن تتشابه أسماء ال variables في ال functions المختلفة.

- Changing the actual parameter.

```
#include <iostream>
using namespace std;

void increment(int &value) {
    value++;
}

int main() {
    int num = 5;
    increment(num);
    cout << "Incremented Value: " << num << endl; // Output: Incremented Value: 6
    return 0;
}
```

في هذا المثال
عدنا على

قيمة num في ال increment function وظهر أثر هذا التعديل في ال main function (اصبحت قيمته في
ال main function تساوي 6 لأنه reference parameter).

- When passing the address would save memory space and time.

أن نرسل ال memory location الى ال formal parameter يقلل عدد ال variables في ال memory ويجعل
البرنامج أسرع.
* للإشارة أن هذا reference parameter نضع بعد ال data type لل parameter '&' .

example:

الأثر الجميل يعمل بسرعة

```

#include <iostream>
using namespace std;

void getScore(int& score);
void printGrade(int score);

int main()
{
    int courseScore;

    cout << "Line 1: Based on the course score, \n"
         << "    this program computes the "
         << "course grade." << endl;           //Line 1

    getScore(courseScore);                     //Line 2

    printGrade(courseScore);                   //Line 3

    return 0;
}

void getScore(int& score)
{
    cout << "Line 4: Enter course score: ";    //Line 4
    cin >> score;                             //Line 5
    cout << endl << "Line 6: Course score is "
         << score << endl;                   //Line 6
}

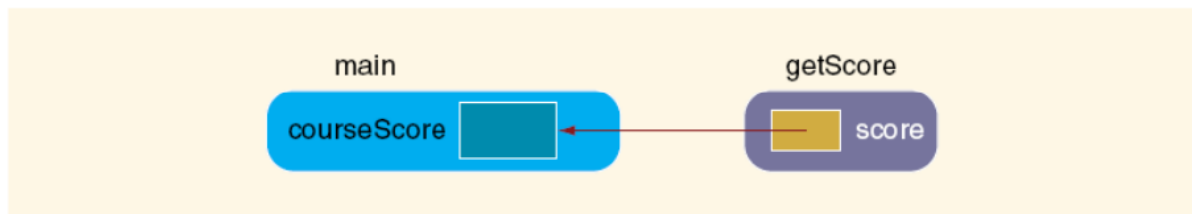
void printGrade(int cScore)
{
    cout << "Line 7: Your grade for the course is "; //Line 7

    if (cScore >= 90)                          //Line 8
        cout << "A." << endl;
    else if (cScore >= 80)
        cout << "B." << endl;
    else if (cScore >= 70)
        cout << "C." << endl;
    else if (cScore >= 60)
        cout << "D." << endl;
    else
        cout << "F." << endl;
}

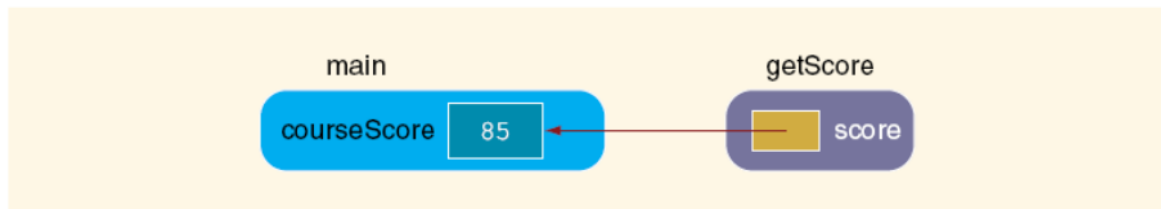
```



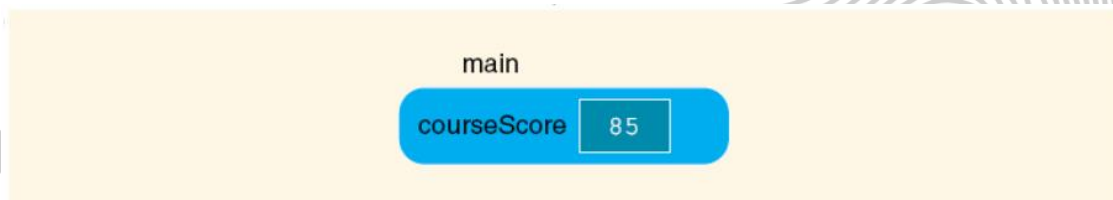
في هذا المثال استخدمنا variable واحد فقط courseScore أخذناه as an input في getScore function ثم أرسلناه إلى printGrade ليقيم بطباعته (بعد أن كانت قيمته التي أدخلت في getScore function خزنت في الـ main function بإسم courseScore).



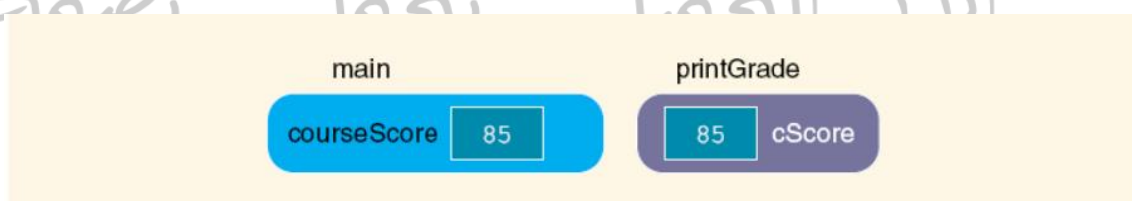
ال formal parameter الذي اسمه score يؤشر على موقع courseScore في ال main function لأنه reference parameter.



فرضا أدخل المستخدم 85 تخزن هذه القيمة في courseScore في ال main وليس في score لأن score هو ال memory location الخاص بـ courseScore.



بعد أن تنفذ جملة cin في line 5 والعودة الى ال main يتم تخزين القيمة التي تم إدخالها في courseScore في ال main.



بعد أن ينفذ الجملة في line 3 يخزن في cScore في ال printGrade function نسخة عن courseScore في ال main function لأنه value parameter.

Value and Reference Parameters and Memory Allocation

- When a function is called :
 - Memory for its formal parameters and variables declared in the body of the function (called local variables) is allocated in the function data area.

يتم تعريف ال variables في ال formal parameter list وحجز لهم مكان في ال memory فقط عندما يتم استدعاء ال function، ال variables الخاصة بكل function تسمى local variables.

- In the case of a value parameter :
 - The value of the actual parameter is copied into the memory cell of its corresponding formal parameter.

في حال كان value parameter قيمة ال actual parameter تنتسخ الى ال memory location الخاص بال formal parameter .

- In the case of a reference parameter :
 - The address of the actual parameter passes to the formal parameter.
 - Content of formal parameter is an address.

في حال كان reference parameter موقع ال actual parameter يعطى الى ال formal parameter، المحتوى المخزن في ال formal parameter هو memory location.

- During execution, changes made by the formal parameter permanently change the value of the actual parameter.

خلال التنفيذ التغيرات التي تطرأ على ال formal parameter تغير قيمة ال actual parameter في ال main.

EXAMPLE 7-7

```
#include <iostream>

using namespace std;

void funOne(int a, int& b, char v);
void funTwo(int& x, int y, char& w);

int main()
{
    int num1, num2;
    char ch;

    num1 = 10; //Line 1
    num2 = 15; //Line 2
    ch = 'A'; //Line 3

    cout << "Line 4: Inside main: num1 = " << num1
         << ", num2 = " << num2 << ", and ch = "
         << ch << endl; //Line 4
    funOne(num1, num2, ch); //Line 5

    cout << "Line 6: After funOne: num1 = " << num1
         << ", num2 = " << num2 << ", and ch = "
         << ch << endl; //Line 6

    funTwo(num2, 25, ch); //Line 7

    cout << "Line 8: After funTwo: num1 = " << num1
         << ", num2 = " << num2 << ", and ch = "
         << ch << endl; //Line 8

    return 0;
}
```

الأثر الجميل يعمل بصمت

```

void funOne(int a, int& b, char v)
{
    int one;

    one = a;           //Line 9
    a++;               //Line 10
    b = b * 2;         //Line 11
    v = 'B';           //Line 12

    cout << "Line 13: Inside funOne: a = " << a
          << ", b = " << b << ", v = " << v
          << ", and one = " << one << endl; //Line 13
}

void funTwo(int& x, int y, char& w)
{
    x++;               //Line 14
    y = y * 2;         //Line 15
    w = 'G';           //Line 16

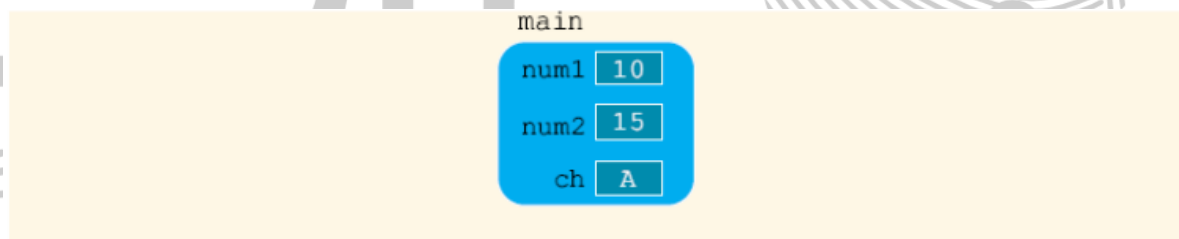
    cout << "Line 17: Inside funTwo: x = " << x
          << ", y = " << y << ", and w = " << w
          << endl;     //Line 17
}

```

Sample Run:

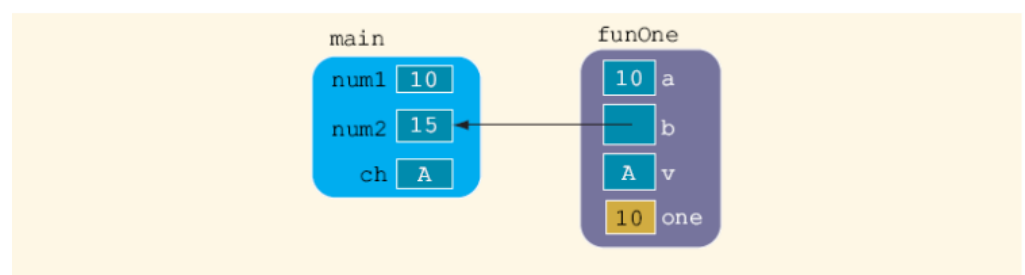
Line 4: Inside main: num1 = 10, num2 = 15, and ch = A
 Line 13: Inside funOne: a = 11, b = 30, v = B, and one = 10
 Line 6: After funOne: num1 = 10, num2 = 30, and ch = A
 Line 17: Inside funTwo: x = 31, y = 50, and w = G
 Line 8: After funTwo: num1 = 10, num2 = 31, and ch = G

لاحظ في line 6 في الـ output قيمة num2 فقط تغيرت لأنه الوحيد أرسل إلى reference parameter.

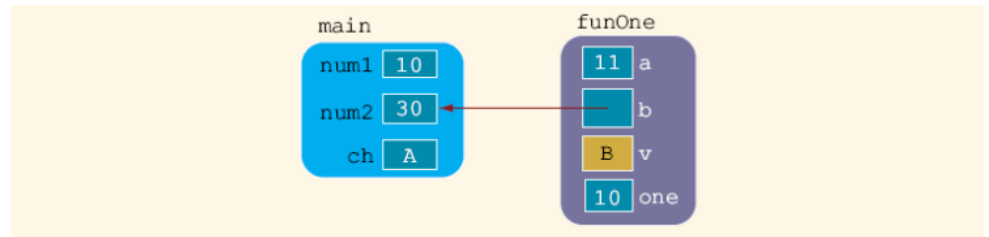


بعد تنفيذ الجملة في line 3.

Line 4: Inside main: num1 = 10, num2 = 15, and ch = A

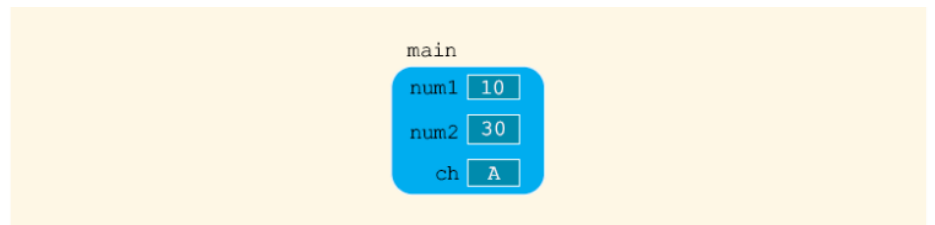


بعد تنفيذ الجملة في line 9.



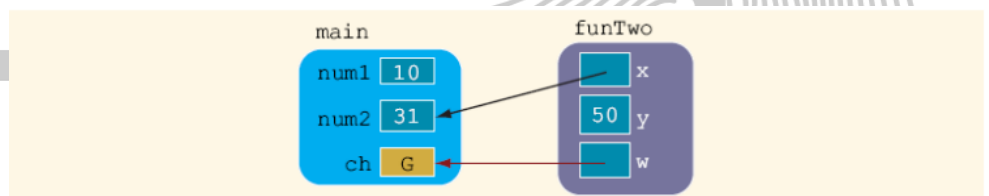
بعد تنفيذ الجمل في line 10,11,12 . (داخل funOne)

Line 13: Inside funOne: a = 11, b = 30, v = B, and one = 10



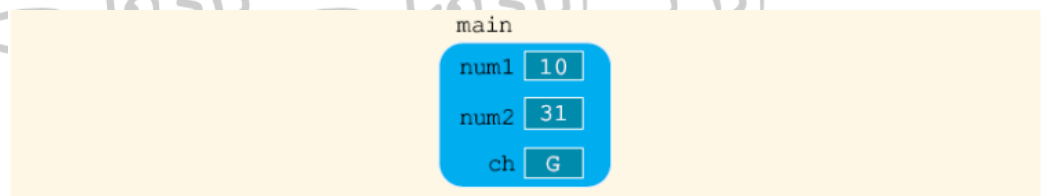
قيمة الvariables في الmain بعد الخروج من funOne.

Line 6: After funOne: num1 = 10, num2 = 30, and ch = A.



بعد تنفيذ الجمل في line 14,15,16 . (داخل funTwo)

Line 17: Inside funTwo: x = 31, y = 50, and w = G



بعد الخروج من funTwo.

Line 8: After funTwo: num1 = 10, num2 = 31, and ch = G

Reference Parameters and ValueReturning Functions

- You can also use reference parameters in a value-returning function
 - Not recommended.

- If a function needs to return more than one value, you should change it to a void function and use the appropriate reference parameters to return the values.

تقدر تستخدم reference parameter في value-returning function لكن لا ينصح به بعمل خريطة، اذا كان الfunction يحتاج أن ترجع أكثر من قيمة حوله الى void function .

Scope of an Identifier

- The scope of an identifier refers to where in the program an identifier is accessible
- ال scope (مجال) ل identifier معين هو الأماكن في البرنامج التي يمكن الوصول فيها الى ال identifier .
- Local identifier: identifiers declared within a function (or block).

ال local identifier هو ال identifier المعرّف ضمن function أو block (بين { }).

مثال: x هو local identifier في ال main function.

```
#include <iostream>

int main()
{
    int x = 1;
    return 0;
}
```

- Global identifier: identifiers declared outside of every function definition.

ال global identifier هو المعرف خارج أي function (ليس ضمن أي function).

```
#include <iostream>

const int pi = 3.14;

int main()
{
    int x = 1;
    return 0;
}
```

مثال: pi هو global identifier .

- C++ does not allow nested functions :
- The definition of one function cannot be included in the body of another function.

ما بنقدر نعرّف function داخل function ثاني.

In general, the following rules apply when an identifier is accessed:

قواعد تذكر متى نستطيع الوصول الى identifier معين:

1. Global identifiers (such as variables) are accessible by a function or a block if:

- The identifier is declared before the function definition (block).
- The function name is different from the identifier.
- All parameters of the function have names different than the name of the identifier
- All local identifiers (such as local variables) have names different than the name of the identifier.

نستطيع الوصول الى global variable اذا كان معرف قبل الfunction، واسمه يجب أن يكون مختلف عن : اسم الfunction و أسماء الparameters و أسماء الlocal identifiers.

```
#include <iostream>
using namespace std;

int main()
{
    cout<<x;
    return 0;
}
const int x = 3.14;
```

في هذا المثال لا نستطيع الوصول الى الglobal variable x لأنه معرف بعد الfunction.

*it is an error.

```
#include <iostream>
using namespace std;

const int x = 3.14;
int main()
{
    int x =1;
    cout<<x;
    return 0;
}
```

في هذا المثال لا نستطيع أن نصل إلى الglobal variable x لأن الamin function لديه variable بنفس الاسم.

الoutput رح يكون 1 (قيمة الlocal identifier)

*it is not an error

```
#include <iostream>
using namespace std;

const int x = 3.14;

int x ()
{
    return 1;
}
```

في هذا المثال لا نستطيع الوصول إلى الـ global variable x لأنه بنفس اسم الـ function.

*it is an error

```
#include <iostream>
using namespace std;

const int x = 3.14;

int sum (int x , int y)
{
    return x+y ;
}
```

في هذا المثال لا نستطيع الوصول إلى الـ global variable x لأنه بنفس اسم parameter في الـ sum function.

*it is an error

An identifier declared within a block is accessible:

- Only within the block from the point at which it is declared until the end of the block.
- By those blocks that are nested within that block if the nested block does not have an identifier with the same name as that of the outside block (the block that encloses the nested block).

الـ local variable نستطيع الوصول إليه فقط بين الـ block المعرف فيه { }

```
#include <iostream>
using namespace std;

int main()
{
    for(int i=1;i<4;i++)
    {
        cout<<i;
    }
    cout<<i;
    return 0;
}
```

في هذا المثال، لا نستطيع الوصول إلى i خارج الـ for loop لأنه غير معرف خارجها. (نستطيع الوصول له فقط في الـ scope المعرف داخله وهو الـ loop).

إذا كان في block داخل block ، نستطيع الوصول إلى identifier معرف في الـ block الخارجي داخل الـ block الداخلي إذا كان الداخلي لا يحتوي على identifier بنفس الاسم.

```
#include <iostream>
using namespace std;

int main()
{
    for(int i=1;i<4;i++)
    {
        cout<<i;
        for(int j=1; j<=i;j++)
        cout<<i;
    }

    return 0;
}
```

في هذا المثال نستطيع أن نصل الى i في for الداخلية لأنها معرفة خارجها ولا تحتوي على identifier بنفس الاسم.

The scope of a function name is similar to the scope of an identifier declared outside any block. That is, the scope of a function name is the same as the scope of a global variable.

الـ functions غير الـ main مثل قواعد الـ global variables نقدر نوصلهم اذا كان تعريفهم فوق الـ function الخ.

- Some compilers initialize global variables to default values.

بعض الـ compilers الـ global variables بيعطوهم default value اذا ما اعطيتهم قيمة. في visual studio المعتمد في هذه المادة الـ default value هي صفر.

- The operator :: is called the scope resolution operator
- By using the scope resolution operator, a global variable declared before the definition of a function (block) can be accessed by the function (or block) even if the function (or block) has an identifier with the same name as the variable .

باستخدام scope resolution operator (::) نستطيع الوصول الى global variable معرّف قبل الـ function حتى لو كان الـ function يحتوي على variable بنفس اسم الـ global variable.


```
#include <iostream>
using namespace std;
int y = 3;
int main()
{
    double y = 2.5;
    cout<<y<<endl;    //line 2
    cout<<::y;        //line 3

    return 0;
}
```

في هذا المثال، ال main function يحتوي على local variable بنفس اسم ال global variable، الجملة في line 2 ستطبع 2.5 (قيمة ال local variable) أما الجملة في line 3 ستطبع 3 (قيمة ال global variable) لأننا استخدمنا (::).

- C++ provides a way to access a global variable declared after the definition of a function:
- In this case, the function must not contain any identifier with the same name as the global variable.

في البرمجة طريقة نستطيع الوصول الى global variable معرّف بعد ال function لكن بشرط أن لا يحتوي ال function على local variable بنفس الاسم. (غير مطالبين بالطريقة)

Global Variables, Named Constants, and Side Effects

- A function that uses global variables is not independent
- If more than one function uses the same global variable and something goes wrong
 - It is difficult to find what went wrong and where
 - Problems caused in one area of the program may appear to be from another area
- Global named constants have no side effects

إذا ظهر عندي error سواء syntax أو logical أثناء استخدام global variable في أكثر من function، صعب معرفة أين الخطأ وتصحيحه لذا من الأفضل استخدام global named constants لأنه ما له سلبيات. كالآتي:

```
#include <iostream>
using namespace std;
const char v= 'F';
```

Static and Automatic Variables

- Automatic variable: memory is allocated at block entry and deallocated at block exit
- By default, variables declared within a block are automatic variables.

الautomatic variable هو اللي يتم حجز له مكان في الmemory عند تعريفه ويتم ازالته عند الخروج من الblock .

```
#include <iostream>
using namespace std;

int main()
{
    {
        int y = 0; //Line 1
        cout<<y;
    } //Line 4

    return 0;
}
```

في هذا المثال عند الدخول في الblock في line 1 عرّفنا ال variable y وطبعناه لكن عند الخروج من الblock في line 4 يتم إزالة y من الmemory يعني لو حاولت ان أستخدم y في جمل خارج الblock سيعطيني error (وهذا ليس بشيء جديد أخذناه في تعريف ال local variables).

```
#include <iostream>
using namespace std;

int main()
{
    for(int i =1; i<=5;i++)
    {
        for(int j = 1; j<=i ; j++)
        {
            cout<<'!';
        }
    }

    return 0;
}
```

طبعا الblock مش شرط يكون محاط ب{} ; في هذا المثال for loop الداخلية هي عبارة عن block وال variable معرف ضمن هذا block أي عند الخروج منه يتم إزالة ال variable من ال memory .

- Static variable: memory remains allocated as long as the program executes.

عكس الautomatic قيمته تبقى مخزنة في الmemory طالما البرنامج ينفذ.
مثال: لو بدنا نحسب كم مرة تم استدعاء function معين :

```
#include <iostream>
using namespace std;

void myFunction() {
    int count = 0;
    count++;
    cout << "Function has been called " << count << " times." << endl;
}

int main() {
    myFunction(); // Call 1
    myFunction(); // Call 2
    myFunction(); // Call 3

    return 0;
}
```

لو استخدمنا automatic variable الذي هو count، أول مرة يتم استدعاء الfunction بزيادة قيمة count بمقدار واحد بس بمجرد ما طلعت من الfunction يتم ازالته من الmemory، في المرة الثانية يرجع بقرأ جملة تعريفه ويزيده واحد والمرة الثالثة نفس الاشئ يعني فعليا ما اعطاني النتيجة الي بدي ياها. عشان نحل هاي المشكلة بنخليه static variable يحتفظ بقيمته طالما البرنامج بتنفذ كالاتي:

```
#include <iostream>
using namespace std;

void myFunction() {
    // Static variable to count the number of times the function is called
    static int count = 0;
    count++;
    cout << "Function has been called " << count << " times." << endl;
}

int main() {
    myFunction(); // Call 1
    myFunction(); // Call 2
    myFunction(); // Call 3

    return 0;
}
```

أول مرة يتم فيها استدعاء الfunction يعرفه ويعطيه قيمة ابتدائية صفر ويزيده بمقدار واحد، بما انه static يحتفظ بقيمته عند الخروج من الfunction، المرة الثانية التي يتم استدعاء الfunction تكون قيمة الcount واحد ويزيدها بمقدار واحد تصبح اثنين، وهكذا في كل مرة يتم فيها استدعاء الfunction.

-Variables declared outside of any block are static variables

الvariables المعرفين خارج أي block هم static by default.

- The syntax for declaring a static variable is:

```
static dataType identifier;
```

- Static variables declared within a block are local to the block, their scope is the same as any other local identifier of that block.

الstatic variables المعرفين ضمن block همه local variables والScope الخاص فيهم مثل أي local variable.

Function Overloading

- In a C++ program, several functions can have the same name.

ما هو الfunction overloading ؟ هو أن يكون هناك أكثر من function بنفس الاسم.

- Two functions are said to have different formal parameter lists if both functions have:

–A different number of formal parameters, or if the number of formal parameters is the same, then the data type of the formal parameters, in the order you list them, must differ in at least one position.

لكن طالما الهم نفس الاسم لازم يختلفوا في الformal parameter list ، اللي ممكن يختلفوا في ثلاث اشياء: عدد الparameters ، أو الdata types of parameters ، أو ترتيب الdata types.

أمثلة:

```
int min (int x, int y, int z)
```

```
int min (int one, int two)
```

الثنتين الهم نفس الاسم ونفس الdata types لكن يختلفوا في عدد الparameters.

```
void funOne (double a, int b)
```

```
void funOne (int i, int j)
```

نفس الاسم ونفس عدد الparameters لكن يختلفوا في الdata type.

```
void fun (int a, double b, char c)
```

```
void fun (char one, double two, int three)
```

نفس الاسم ونفس العدد ونفس ال data types لكن يختلفوا في ترتيب ال data types.

يعني لازم يختلفوا في وحدة من الثلاث اشياء.

```
int function (int x, double y)
```

```
void function (int a, double y)
```

هدول نفس كلشي هاد syntax error .

- The signature of a function consists of the function name and its formal parameter list.

ال signature الخاص ب function معين هو اسمه + formal parameter list . (لاحظوا انه بدون نوع ال function)
مثال:

```
min (int x, int y)
```

عشان يختلف ال signature ل two functions لازم يختلفوا بالاسم أو ال formal parameter list.

Functions with Default Parameters

- In a function call, the number of actual and formal parameters must be the same
-C++ relaxes this condition for functions with default parameters

في ال function call يجب أن يكون عدد ال actual parameters اللي بنرسلهم نفس عدد ال formal parameter ، لكن في حال كان عنا default parameters ممكن نرسل عدد أقل.

- You specify the value of a default parameter when the function name appears for the first time (e.g., in the prototype), If you do not specify the value of a default parameter, the default value is used.

نحدد ال default parameter داخل ال formal parameter list كالاتي:

```
int min (int x, int y, int z = 0)
```

إذا ما حددنا قيمة لل parameter الثالث في ال function call بيستخدم default value اللي هي صفر مثل الاتي:
min(x, 4);

- All default parameters must be the rightmost parameters of the function

تحديد default parameter لازم يبلش من اليمين الى اليسار:

```

int min (int x, int y, int z = 0)  اوك
int min (int x, int y=0, int z = 0)  اوك
int min (int x=0, int y=1, int z = 0)  اوك
int min (int x, int y=0, int z )  مش اوك
int min (int x=0, int y, int z )  مش اوك
int min (int x=0, int y=1, int z )  مش اوك
void fun ( int a , int b=2, int c, int d=1)  مش اوك
int fun (int x =1)  اوك
  
```

- Default values can be constants, global variables, or function calls. However, you cannot assign a constant value as a default value to a reference parameter.

ال default values بتقدر تحددوها شو ما بدك constant, global variable, function call لكن في حال كان reference parameter ما بتقدر تحدد اله constant default value (بكل بساطة لأنه يستقبل memory location).

```

void (int& x =3)  مش اوك
  
```

Problem-Solving Part

1. Write a C++ program that inputs two integers and define a function that swaps these two integers.
2. Write a C++ program that reads three integers from the user and determines the maximum of the three numbers using a void function that updates the first argument to hold the maximum value. Then, print the maximum value in the main() function.
3. Write a C++ program that counts the number of arguments sent to a function (the function has one argument).

4. Write a C++ program that calculates the volume of different shapes (a cube and a rectangular box). Use function overloading to handle the different shapes and default parameters to provide default values for the dimensions.

1.

```
#include <iostream>
using namespace std;

void swap (int &a, int& b)
{
    int flag=a;
    a=b;
    b=flag;
}

int main() {
    int num1, num2;
    cout<<"Enter two integers";
    cin>>num1>>num2;
    cout<<"Before swap : num1 = "<<num1<<", num2 = "<<num2<<endl;
    swap(num1,num2);
    cout<<"After swap : num1 = "<<num1<<", num2 = "<<num2;

    return 0;
}
```

2.


```

#include <iostream>
using namespace std;

void max (int &a, int b, int c)
{
    if(b>a)
        a=b;
    if(c>a)
        a=c;
}

int main() {

    int x,y,z;
    cout<<"Enter three integers";
    cin>>x>>y>>z;
    max(x,y,z);
    cout<<endl<<"The maximum number is : "<<x;
    return 0;
}

```

3.

```

#include <iostream>

void accumulateSum(int number) {
    // Static variable to keep track of the total sum
    static int totalSum = 0;
    totalSum += number;
    std::cout << "Current number: " << number << ", Total sum: " << totalSum << std::endl;
}

int main() {
    accumulateSum(5);    // Call 1
    accumulateSum(10);   // Call 2
    accumulateSum(3);    // Call 3

    return 0;
}

```

4.

```
#include <iostream>
using namespace std;

// Function to calculate the volume of a cube
int volume(int side) {
    return side * side * side;
}

// Function to calculate the volume of a rectangular box with default parameters
int volume(int length, int width, int height = 1) {
    return length * width * height;
}

int main() {
    // Volume of a cube with side length 3
    cout << "Volume of cube with side 3: " << volume(3) << endl;

    // Volume of a rectangular box with length 3, width 4, and default height 1
    cout << "Volume of box with length 3, width 4: " << volume(3, 4) << endl;

    // Volume of a rectangular box with length 3, width 4, and height 5
    cout << "Volume of box with length 3, width 4, height 5: " << volume(3, 4, 5) << endl;

    return 0;
}
```



الأثر الجميل يعمل بصمت